

Técnicas de Priorização Ágil no Desenvolvimento de Software: Uma Revisão Teórica e Aplicações Práticas

Valdir S. C. Neto¹, Patrícia S. Costa¹, Camila P. Fernandes¹

¹Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais — Campus São
João Evangelista (IFMG)
39.705-000 – São João Evangelista – MG – Brazil

{0082926,0082931,0082934}@academico.ifmg.edu.br

Abstract. *This text investigates the essential prioritization methods employed in agile program development contexts. Having the skill to define what will be built first is crucial to ensuring a constant supply of benefits and minimizing risks. Methods such as MoSCoW, Benefit-Cost Ratio, WSJF, Kano, Scorecard, Story Map, and the Minimum Viable Product (MVP) concept are analyzed from a theoretical perspective. In addition, the work puts three of these strategies into practice in real-world situations, showing how these resources help teams make important choices in program development initiatives.*

Resumo. *Este texto investiga os métodos essenciais de priorização empregados em contextos ágeis de criação de programas. Ter a destreza de definir o que será construído primeiramente é crucial para assegurar a oferta constante de benefícios e a diminuição de perigos. Métodos como MoSCoW, Relação Benefício-Custo, WSJF, Kano, Scorecard, Story Map e a ideia de Produto Mínimo Viável (MVP) são analisados sob um ângulo teórico. Em acréscimo, o trabalho põe em prática três dessas estratégias em situações reais, mostrando como esses recursos ajudam os times a fazer escolhas importantes em iniciativas de construção de programas.*

1. Introdução

O atual cenário do desenvolvimento de software exige adaptação contínua e foco exacerbado na entrega de valor. Projetos extensos, construídos sob paradigmas tradicionais e com especificações engessadas, frequentemente culminam em produtos que não atendem às necessidades reais dos usuários ou sofrem com o desperdício de recursos [Sommerville 2007].

Nesse contexto de incertezas e orçamentos limitados, a priorização emerge não apenas como uma ferramenta de gestão, mas como um mecanismo vital de sobrevivência e vantagem competitiva.

1.1. Problema

A problemática central na gestão de produtos reside na abundância de requisitos versus a escassez de tempo e capacidade de engenharia. Quando stakeholders são solicitados a elencar funcionalidades, existe a tendência natural de que tudo seja considerado como prioridade máxima, gerando *backlogs* inflados e inatingíveis [Cohn 2005]. A ausência de uma priorização metódica leva ao desenvolvimento de *features* de baixo impacto, afetando a qualidade geral do sistema e resultando em retrabalho, atrasos e insatisfação dos clientes [Ries 2011].

1.2. Justificativa

Justifica-se, portanto, a utilização de *frameworks* e técnicas ágeis de priorização. Tais abordagens oferecem modelos estruturados que auxiliam na separação do que é essencial do que é apenas desejável, alinhando a produção aos objetivos de negócio e reduzindo o *time-to-market* [Caroli 2024]. Além disso, promovem transparência nas decisões e facilitam a comunicação entre equipes técnicas e partes interessadas.

1.3. Objetivo

O objetivo deste artigo é apresentar uma revisão teórica das técnicas MoSCoW, Valor x Esforço, *Weighted Shortest Job First* (WSJF), Kano, Scorecard, *Story Map* e Produto Mínimo Viável (MVP), demonstrando sua aplicabilidade prática por meio de três cenários simulados. Busca-se fornecer um guia conciso para profissionais de produto e desenvolvimento que desejam aprimorar seus processos de priorização.

1.4. Relevância

Compreender e saber aplicar diferentes métodos de priorização é competência central para *Product Owners*, *Product Managers* e equipes ágeis. Este trabalho contribui ao consolidar o conhecimento disperso na literatura e ao exemplificar o uso das técnicas em contextos realísticos, facilitando a transferência de teoria para a prática.

1.5. Organização do trabalho

O artigo está organizado da seguinte forma: a Seção 2 apresenta a revisão da literatura com a fundamentação teórica de cada técnica. A Seção 3 detalha a aplicação prática de três métodos em cenários criados. Por fim, a Seção 4 traz as considerações finais e conclusões.

2. Revisão da literatura

A engenharia de requisitos moderna adaptou inúmeras abordagens para lidar com o desafio de priorizar tarefas [Vestola 2011]. A seguir, exploram-se as características teóricas das metodologias propostas.

2.1. MoSCoW

A técnica MoSCoW, originada no *Dynamic Systems Development Method* (DSDM) por Dai Clegg na Oracle, classifica os requisitos em quatro categorias [Agile Business Consortium 2014]. O acrônimo representa:

- **Must have:** requisitos obrigatórios, sem os quais o produto não é viável ou o projeto seria cancelado. Devem ocupar, no máximo, 60% do esforço total de desenvolvimento para garantir contingência [Miranda 2011].
- **Should have:** itens importantes, mas não vitais. Sua ausência pode ser contornada com soluções alternativas temporárias.
- **Could have:** desejáveis, porém de baixo impacto se omitidos. Constituem a principal reserva de contingência.
- **Won't have:** itens explicitamente excluídos do escopo atual, mas que podem ser reconsiderados futuramente.

A principal força do MoSCoW é a clareza que proporciona às partes interessadas, estabelecendo expectativas realistas sobre o que será entregue e o que ficará para versões posteriores [Agile Business Consortium 2026]. Contudo, sua natureza qualitativa pode levar a subjetividade se não houver critérios objetivos para classificação.

2.2. Valor x Esforço (Value vs. Effort)

Também conhecida como Matriz de Priorização Lean ou Matriz Impacto versus Esforço, esta técnica posiciona os itens em uma grade 2x2 com eixos de Valor (ou Impacto) e Esforço (ou Complexidade) [Cohn 2005]. Os quadrantes resultantes orientam a ação:

- **Quick Wins** (Alto Valor, Baixo Esforço): prioridade máxima; devem ser executados imediatamente.
- **Major Projects** (Alto Valor, Alto Esforço): projetos estratégicos que exigem planejamento cuidadoso e podem ser fatiados.
- **Fill-ins** (Baixo Valor, Baixo Esforço): tarefas que preenchem lacunas de tempo ocioso.
- **Time Wasters** (Baixo Valor, Alto Esforço): itens a serem evitados ou eliminados.

A simplicidade é seu maior benefício, permitindo rápida visualização e alinhamento da equipe. Por outro lado, a definição de “valor” é subjetiva e pode variar conforme o avaliador [Highberg 2023]. Recomenda-se combiná-la com outras técnicas quando maior precisão for necessária.

2.3. Weighted Shortest Job First (WSJF)

O WSJF é um modelo quantitativo amplamente adotado no *Scaled Agile Framework* (SAFe) para priorizar *features* e épicos [Scaled Agile, Inc. 2023]. Ele calcula um escore baseado no Custo do Atraso (*Cost of Delay* - CoD) dividido pelo tamanho do trabalho (*Job Size*):

$$WSJF = \frac{BusinessValue + TimeCriticality + RiskReduction}{JobSize}$$

Os componentes do numerador são avaliados em escalas relativas (ex.: Fibonacci modificado 1,2,3,5,8,13). O denominador representa o esforço estimado. Quanto maior o WSJF, maior a prioridade.

O WSJF é particularmente útil para equilibrar iniciativas de curto prazo (alto valor, baixo esforço) e investimentos estratégicos, além de incorporar a urgência e a redução de riscos [SixSigma.us 2024]. Sua principal desvantagem é a necessidade de estimativas confiáveis e o envolvimento de múltiplos *stakeholders* na pontuação.

2.4. Modelo Kano

Desenvolvido por Noriaki Kano na década de 1980, este modelo classifica funcionalidades com base no impacto que causam na satisfação do cliente [Kano et al. 1984]. As categorias são:

- **Obrigatórias (Must-be)**: características básicas que o cliente espera. Quando presentes, não geram satisfação adicional; quando ausentes, causam forte insatisfação.
- **Lineares (One-dimensional)**: quanto mais bem implementadas, maior a satisfação. Exemplo: velocidade de carregamento.
- **Atrativas (Delighters)**: elementos surpresa que encantam o usuário. Sua ausência não gera insatisfação, mas sua presença eleva a satisfação acima do esperado.

-
- **Indiferentes e Reversas:** itens que não impactam ou causam efeito negativo.

A aplicação do Kano requer a aplicação de um questionário com perguntas funcionais (“Como você se sentiria se o recurso estivesse presente?”) e disfuncionais (“... se não estivesse presente?”) [Tera 2025]. A técnica é poderosa para orientar *roadmaps* centrados no usuário, mas demanda tempo e acesso a clientes reais.

2.5. Scorecard

O Scorecard de Priorização é uma abordagem quantitativa na qual a equipe define critérios ponderados (ex.: impacto no negócio, urgência, custo, risco técnico) e atribui notas a cada item do *backlog* [Cagan 2009]. O escore final de cada item é calculado tipicamente como uma média ponderada ou soma dos critérios.

$$Prioridade = \sum_{i=1}^n (Peso_i \times Nota_i)$$

Os itens são então ordenados por pontuação decrescente. O Scorecard promove transparência e decisões baseadas em dados, reduzindo vieses. Pode ser personalizado para diferentes contextos (ex.: adicionar “confiança” ou “alinhamento estratégico”) e é frequentemente integrado a ferramentas de gestão de produto como Aha! [Aha! Labs 2024]. A desvantagem é a complexidade inicial para definir pesos e escalas consensuais.

2.6. Story Map

Proposto por Jeff Patton, o *User Story Mapping* organiza requisitos visualmente para contar a história da jornada do usuário com o sistema [Patton 2014]. A estrutura é bidimensional:

- **Eixo horizontal (backbone):** atividades principais que o usuário realiza, em ordem cronológica.
- **Eixo vertical:** detalhamento das tarefas, *epics* e histórias de usuário, organizadas por prioridade.

A partir do mapa, a equipe traça “cortes” horizontais (fatias) que representam versões incrementais do produto, sendo a primeira fatia o MVP. O *Story Map* ajuda a evitar o problema de construir funcionalidades isoladas que não formam um todo coeso [PM3 2020]. É uma ferramenta colaborativa excelente para *product discovery* e planejamento de *releases*.

2.7. Produto Mínimo Viável (MVP)

Embora mais do que uma técnica de priorização, o conceito de MVP é central para a metodologia *Lean Startup* de Eric Ries [Ries 2011]. MVP é a versão de um produto que permite completar o ciclo *Construir-Medir-Aprender* com o mínimo de esforço e tempo. Ele contém apenas as funcionalidades estritamente necessárias para validar hipóteses de negócio e obter *feedback* real de usuários.

A definição do MVP está intrinsecamente ligada à priorização: é preciso decidir quais *features* são indispensáveis para o aprendizado e quais podem ser postergadas [Caroli 2024]. Erros comuns incluem construir um MVP muito complexo (perde a agilidade) ou muito pobre (não entrega valor mínimo). Exemplos célebres incluem o vídeo de demonstração do Dropbox e o site inicial da Zappos, que operava com estoque comprado após os pedidos.

3. Aplicação Prática das Técnicas

Para demonstrar a aplicabilidade das técnicas, foram criados três cenários fictícios de desenvolvimento de software. Em cada um, aplica-se uma técnica diferente, detalhando o passo a passo e os resultados obtidos.

3.1. Cenário 1: Priorização de Funcionalidades com MoSCoW em um App de Delivery

Contexto: Uma startup está desenvolvendo um aplicativo de delivery de refeições chamado “FoodExpress”. O time de produto possui um *backlog* inicial com 15 funcionalidades, mas a primeira versão (MVP) precisa ser lançada em 8 semanas com recursos limitados (4 desenvolvedores, 1 designer, 1 QA). O *Product Owner* decide usar MoSCoW.

Aplicação:

1. Listagem de todos os requisitos elicitados junto a potenciais usuários e *stakeholders*.
2. Reunião de classificação com a equipe e representantes do negócio. Para cada item, pergunta-se: “O produto faria sentido sem isso? O lançamento seria cancelado se não entregássemos?”
3. Resultado da classificação (exemplo parcial):
 - **Must have:** cadastro/login (e-mail ou telefone), busca de restaurantes por categoria, visualização de cardápio, adição ao carrinho, finalização de pedido (com pagamento simulado), notificação de status do pedido (básica).
 - **Should have:** avaliação de restaurantes e pratos, histórico de pedidos, cálculo de frete por distância.
 - **Could have:** programa de fidelidade, recomendação baseada em compras anteriores, integração com redes sociais.
 - **Won’t have:** pagamento por criptomoedas, modo escuro customizável, suporte por chat ao vivo.
4. A equipe estima o esforço (em story points) e verifica que os *Must have* somam aproximadamente 55% da capacidade da *sprint*. Dentro do limite recomendado, o MVP é definido contendo apenas os *Must have*. Os *Should have* serão planejados para a segunda versão.

Resultado: O MoSCoW trouxe clareza imediata sobre o escopo do MVP, evitando que a equipe desperdiçasse tempo em funcionalidades de baixa prioridade. Houve consenso entre os *stakeholders* sobre o que era realmente obrigatório.

3.2. Cenário 2: Aplicação do Modelo Kano para uma Plataforma de Cursos Online

Contexto: Uma edtech já possui uma plataforma de cursos sob demanda (modelo básico). A equipe de produto deseja lançar uma nova versão com funcionalidades inovadoras, mas precisa entender quais itens trarão maior satisfação aos alunos. Opta-se pelo Modelo Kano.

Aplicação:

-
1. Seleção de 8 funcionalidades candidatas: (1) certificado digital automático, (2) legendas automáticas em vídeos, (3) fórum de dúvidas moderado por professores, (4) gamificação com badges, (5) aplicativo móvel com download offline, (6) recomendação de cursos baseada em perfil, (7) acesso vitalício ao conteúdo, (8) suporte por chat 24h.
 2. Elaboração de um questionário online com perguntas funcionais e disfuncionais para cada item. Exemplo para “legendas automáticas”: “Como você se sentiria se a plataforma oferecesse legendas automáticas em todos os vídeos?” (opções: gosto/espero/neutro/tolerar/não gosto) e “Como se sentiria se NÃO oferecesse legendas automáticas?”.
 3. Envio do questionário para 50 alunos ativos e 30 potenciais.
 4. Consolidação das respostas por meio da matriz de avaliação Kano. Os resultados indicaram:
 - **Obrigatórias:** certificado digital automático, acesso vitalício. Sua ausência geraria forte insatisfação.
 - **Lineares:** aplicativo offline, fórum moderado. Quanto melhor implementados, maior a satisfação.
 - **Atrativas:** legendas automáticas, recomendação personalizada. Surpreendem positivamente, mas não são esperadas.
 - **Indiferentes:** gamificação com badges. Não impactou significativamente.
 - **Reversa:** suporte 24h via chat (apenas para público corporativo, não para alunos individuais).
 5. A equipe decidiu priorizar as Obrigatórias e Lineares para o próximo trimestre, planejou uma *feature* atrativa (legendas) para o segundo trimestre como diferencial competitivo, e descartou gamificação para o momento.

Resultado: O Kano evitou o erro de investir em funcionalidades indiferentes ou reversas. O time ficou alinhado sobre o que realmente encanta os usuários (legendas automáticas) e o que é básico de sucesso (certificado).

3.3. Cenário 3: Priorização WSJF em um Projeto de Evolução de E-commerce

Contexto: Uma empresa de médio porte possui uma plataforma de e-commerce consolidada e deseja implementar melhorias em seu fluxo de checkout para reduzir abandono de carrinho. O backlog contém 6 iniciativas. A equipe de produto usa SAFe e decide aplicar WSJF para ordenar as entregas no próximo *Program Increment* (PI).

Aplicação:

1. Lista das iniciativas: A – checkout em uma página (reduzir etapas); B – integração com Pix e carteiras digitais; C – cálculo automático de frete por CEP sem recarregar página; D – recomendação de produtos complementares no carrinho; E – abandono de carrinho recuperado via e-mail automático; F – suporte ao vivo no checkout.
2. Reunião de priorização com *Product Manager*, *System Architect* e *stakeholders* de negócio. Cada item recebe pontuação (Fibonacci 1,2,3,5,8,13) para *User/Business Value* (BV), *Time Criticality* (TC), *Risk Reduction* (RR) e *Job Size* (JS).
3. Cálculo do WSJF = $(VB+TC+RR) / JS$. Resultados (exemplo):

Iniciativa	BV	TC	RR	JS	WSJF
A (checkout 1 página)	8	5	3	5	$(8+5+3)/5 = 3,2$
B (Pix)	13	8	5	8	$(13+8+5)/8 = 3,25$
C (frete automático)	5	3	2	2	$(5+3+2)/2 = 5,0$
D (recomendações)	3	2	1	3	2,0
E (recuperação carrinho)	5	5	4	3	4,66
F (suporte ao vivo)	2	1	3	5	1,2

- Ordem final de prioridade: C (frete automático) > E (recuperação e-mail) > B (Pix) > A (checkout 1 página) > D (recomendações) > F (suporte ao vivo).
- O time percebeu que a iniciativa C, embora de médio valor de negócio, tem altíssima criticidade de tempo (concorrentes já oferecem) e baixo esforço, tornando-a a mais prioritária. A equipe planeja entregar C e E no primeiro *sprint* do PI.

Resultado: O WSJF permitiu uma decisão baseada em dados econômicos, não apenas em opiniões. O item de menor esforço e alta urgência foi corretamente alçado ao topo, maximizando o valor entregue no período.

4. Conclusão

As metodologias ágeis de priorização constituem um conjunto indispensável de ferramentas para equipes de software que buscam entregar valor de forma consistente em ambientes de alta incerteza e recursos limitados. Este artigo revisou sete técnicas distintas – MoSCoW, Valor x Esforço, WSJF, Kano, Scorecard, Story Map e MVP – destacando seus fundamentos, vantagens e limitações. A aplicação prática de três delas (MoSCoW, Kano e WSJF) em cenários realísticos demonstrou como cada técnica pode ser operacionalizada e quais insights específicos proporcionam.

Conclui-se que não existe uma técnica universalmente superior; a escolha depende do contexto organizacional, da maturidade da equipe, do acesso a dados de clientes e do horizonte de planejamento. Em geral, recomenda-se combinar métodos qualitativos (como MoSCoW e Story Map) com métodos quantitativos (como WSJF e Scorecard) e com a visão centrada no usuário do Modelo Kano. Além disso, o conceito de MVP deve guiar a estratégia de lançamento, garantindo aprendizado rápido e redução de desperdício.

Como trabalhos futuros, sugere-se a investigação de *frameworks* híbridos que integrem inteligência artificial para auxiliar na pontuação automática de critérios, bem como estudos de caso longitudinais que meçam o impacto da adoção dessas técnicas no sucesso comercial de produtos de software.

Referências

- Agile Business Consortium (2014). Manual do framework dsdm. Agile Business Consortium.
- Agile Business Consortium (2026). Estrutura do projeto dsdm: Capítulo 10 – priorização do moscow. Agile Business Consortium Limited.
- Aha! Labs (2024). Product value scorecard. Aha! Roadmaps Support.
- Cagan, M. (2009). The product scorecard. Silicon Valley Product Group. Disponível em: <https://svpg.com/product-scorecard/>.

-
- Caroli, P. (2024). Mvp: Conheça e saiba como usar o produto mínimo viável. Caroli.org. Disponível em: <https://caroli.org/mvp-produto-minimo-viavel/>. Acesso em: 09 maio 2026.
- Cohn, M. (2005). *Agile Estimating and Planning*. Pearson Education, Boston.
- Highberg (2023). Value vs. effort matrix: Lean approach. Highberg Insights. Disponível em: <https://highberg.com>.
- Kano, N., Seraku, N., Takahashi, F., and Tsuji, S. (1984). Attractive quality and must-be quality. *Journal of the Japanese Society for Quality Control*, 14(2):39–48.
- Miranda, E. (2011). Time boxing planning: buffered moscow rules. *ACM SIGSOFT Software Engineering Notes*, 36(6):1–5.
- Patton, J. (2014). *User Story Mapping: Discover the Whole Story, Build the Right Product*. O'Reilly Media, Inc., Sebastopol.
- PM3 (2020). O que é user story mapping? PM3 Blog. Disponível em: <https://pm3.com.br/blog/o-que-e-user-story-mapping/>.
- Ries, E. (2011). *A Startup Enxuta*. Lua de Papel, Rio de Janeiro.
- Scaled Agile, Inc. (2023). Weighted shortest job first (wsjf). SAFe Framework. Disponível em: <https://www.scaledagileframework.com/wsjf/>.
- SixSigma.us (2024). Wsjf (weighted shortest job first) – agile framework. SixSigma.us Articles.
- Sommerville, I. (2007). *Engenharia de Software*. Pearson Addison-Wesley, São Paulo, 8 edition.
- Tera (2025). Modelo de kano: o que é? como usar para priorizar o que importa. Tera Blog.
- Vestola, M. (2011). A comparison of nine basic techniques for requirements prioritization.